BTSM — Task Scheduler Setup

This sets up the unattended daily pipeline. Two scheduled tasks total:

Task	When	What it does
`BTSM-Daily-Download`	Once daily @ 10:00 AM ET	Downloads new DRF files from Brisnet
`BTSM-Pipeline-Poller`	Every 30 min, 10:30 AM – 8:00 PM ET	Publishes preview sheets immediately, final sheets 30–60 min before each track's first post

Both tasks email you on failure.

Step 1 — Drop these files into your FullAutomation folder

Save all of these to `C:\Users\ryanr\Documents\BTSM\FullAutomation\`:

- `run_pipeline.py` — the orchestrator (preview + final logic)
- `notify.py` — Gmail SMTP sender
- `run_download.bat` — wrapper for the 10 AM download
- `run_pipeline.bat` — wrapper for the 30-min poller
- `pipeline_state.json` — initial empty state file
- `BTSM-Daily-Download.xml` — Task Scheduler import file
- `BTSM-Pipeline-Poller.xml` — Task Scheduler import file

The downloader (`brisnet_download_v3.py`) is already there from the previous step.

Step 2 — Create a Gmail App Password

1. Go to https://myaccount.google.com — make sure **2-Step Verification is ON** (App Passwords are only available with 2FA enabled).
2. Go to https://myaccount.google.com/apppasswords
3. **App name:** `BTSM Pipeline` → click **Create**
4. Copy the 16-character password (e.g. `abcd efgh ijkl mnop`).
5. Open your existing `.env` file and append these lines:

```
NOTIFY_FROM=youraddress@gmail.com
NOTIFY_TO=youraddress@gmail.com
NOTIFY_APP_PASSWORD=abcd efgh ijkl mnop
```

`NOTIFY_FROM` and `NOTIFY_TO` can be the same (sending alerts to yourself) or different (e.g., `NOTIFY_TO=mobile-text-alias@txt.att.net` for SMS-style alerts).

Step 3 — Test the email setup

From a `cmd` window in the FullAutomation folder:

```
cmd

python notify.py "Test alert" "If you got this, email is working."
```

You should get an email within ~30 seconds. If it fails:

- Common issue: 2-Step Verification not enabled → App Passwords aren't visible
- Common issue: extra spaces in the App Password → `notify.py` strips them, but check `.env` is saved correctly

---

Step 4 — Test the pipeline poller manually

Before scheduling, run it once to verify the orchestrator works:

```
cmd

python run_pipeline.py
```

Expected output:

- Logs `N DRF files found in DRF_Downloads`
- For each DRF: either runs a stub action or skips with a reason
- Writes `pipeline_state.json` (now populated)
- Exit code 0

Open `pipeline_state.json` after the run — you should see entries under `"published"` for each DRF that triggered a preview action. The stub implementations log `[stub]` lines but don't

actually score/PDF/upload yet — those get filled in as we build the next pipeline stages.

Step 5 — Import the Task Scheduler tasks

Easy way: import the XMLs

1. Open **Task Scheduler** (`taskschd.msc`)
2. In the left pane, right-click **Task Scheduler Library** → **New Folder...** → name it `BTSM`
3. Click into the new `BTSM` folder
4. **Action menu → Import Task...**
5. Pick `BTSM-Daily-Download.xml` → **OK**
6. When prompted, **review the user/password dialog** — pick **"Run only when user is logged on"** (simpler) or "Run whether user is logged on or not" (works when locked, but requires entering your password)
7. Repeat **Action → Import Task...** for `BTSM-Pipeline-Poller.xml`

Verify the imported tasks:

- Both should show in the BTSM folder with **Status: Ready**
- **Triggers tab** for the poller should show "At 10:30 AM — Repeat every 30 minutes for a duration of 9 hours and 30 minutes"
- **Actions tab** should point at the right `.bat` file

Step 6 — Run-on-demand test

Right-click each task → **Run**. Watch:

- For `BTSM-Daily-Download`: should produce/update files in `DRF_Downloads/`
- For `BTSM-Pipeline-Poller`: should update `pipeline_state.json` and create `pipeline_run.log`

After running, check **Task Scheduler → History** tab to confirm exit code 0.

How the schedule works in practice

```
Time      | What runs                                | What happens
----------|------------------------------------------|---------------------------
10:00 AM  | BTSM-Daily-Download                      | Pulls today's + future DRFs
10:30 AM  | BTSM-Pipeline-Poller (1st tick)          | Publishes PREVIEW for every
          |                                          |   DRF without one yet
11:00 AM  | BTSM-Pipeline-Poller (2nd tick)          | Republishes PREVIEW for any
          |                                          |   updated DRFs. Checks if
          |                                          |   any track is in 30-60
          |                                          |   min FINAL window
11:30 AM  | BTSM-Pipeline-Poller (3rd tick)          | (Gulfstream first post
          |                                          |   ~12:35 — FINAL fires)
12:00 PM  | poller tick                              | (Belmont/Churchill ~1:00 PM
          |                                          |   first post — FINAL fires)
... continues every 30 min until 8:00 PM ...
```

Idempotency means: if the laptop sleeps at 11:30 and wakes at 12:00, the 12:00 tick catches up the missed 11:30 work. Nothing publishes twice because `pipeline_state.json` tracks what's done.

Common gotchas

1. **"Run only when user is logged on"** vs **"Run whether user is logged on or not"**
   - "Logged on" is simpler — no password prompt. Works as long as you stay logged into Windows. If you log out, tasks pause.
   - "Whether logged on" runs even when you log out, but needs your Windows password stored (it's encrypted by Windows DPAPI). Choose this if your PC is set to lock/log out automatically.
2. **Sleep/hibernate.** The XMLs include `<WakeToRun>true</WakeToRun>` so the PC wakes itself for tasks. But this only works if Windows power settings allow wake timers. Check: `Control Panel → Power Options → Change plan settings → Change advanced power settings → Sleep → Allow wake timers → On`.
3. **Path issues.** If you move the FullAutomation folder, edit the XMLs: the `<Command>` and `<WorkingDirectory>` paths are hard-coded. Change them and re-import.
4. **Python not on PATH.** If `python` isn't in your system PATH, the .bat files will fail. Test with `cmd → python --version`. If it errors, find your Python install (usually

(`C:\Users\ryanr\AppData\Local\Programs\Python\Python3xx\python.exe`) and replace `python` with the full path in both `.bat` files.

5. **Antivirus blocking SMTP.** Some AV products quarantine outgoing SMTP traffic from non-mail-client processes. If `notify.py` test fails with a connection timeout, check your AV's "outbound mail protection" settings.

Monitoring day-to-day

- `pipeline_state.json` — human-readable, shows what's published per track per date
- `pipeline.log` — rolling log of every tick decision (useful for "why didn't track X publish?")
- `pipeline_run.log` — last tick's stdout/stderr (overwritten each run)
- `download_run.log` — last download run's output
- Task Scheduler **History** tab — exit codes and run times for every fired task

If a track's preview never publishes, check `pipeline.log` for `[skip preview]` lines — the reason will be there.

If a track's final never publishes, look for `[skip final]` reasons — usually one of:

- "could not parse first post" (DRF schema offset mismatch — see TODO in `run_pipeline.py:get_first_post`)
- "too late" (the 30-60 min window already closed before any tick caught it)